

L'Apprendimento per Rinforzo: Teoria, PPO e Applicazione al Progetto Snake

Relazione Tecnica

Indice

1	Introduzione al Machine Learning e Reinforcement Learning	2
1.1	Formalizzazione: Processo di Decisione di Markov (MDP)	2
2	L'Algoritmo PPO (Proximal Policy Optimization)	2
2.1	La Funzione Obiettivo di PPO	2
3	Il Progetto Snake RL: Analisi dell'Implementazione	2
3.1	Analisi dei File Sorgenti	3
4	Parametri del Modello e Statistiche	3

1 Introduzione al Machine Learning e Reinforcement Learning

Il Machine Learning (Apprendimento Automatico) è una branca dell'Intelligenza Artificiale che permette ai sistemi di imparare dai dati. Si divide principalmente in tre categorie: Supervised Learning (apprendimento supervisionato), Unsupervised Learning (apprendimento non supervisionato) e Reinforcement Learning (Apprendimento per Rinforzo, RL).

Nel Reinforcement Learning, un **Agente** apprende a prendere decisioni interagendo con un **Ambiente**. L'agente osserva lo stato dell'ambiente, esegue un'azione e riceve una ricompensa (reward) numerica. L'obiettivo è massimizzare la somma delle ricompense nel tempo.

1.1 Formalizzazione: Processo di Decisione di Markov (MDP)

Un problema di RL è tipicamente formalizzato come un Processo di Decisione di Markov (MDP), definito da una tupla (S, A, P, R, γ) :

- S : Spazio degli stati.
- A : Spazio delle azioni.
- $P(s'|s, a)$: Probabilità di transizione allo stato s' partendo dallo stato s ed eseguendo l'azione a .
- $R(s, a)$: Funzione di reward, che restituisce la ricompensa attesa.
- $\gamma \in [0, 1]$: Fattore di sconto (discount factor), che determina l'importanza delle ricompense future rispetto a quelle immediate.

Il ritorno atteso G_t al tempo t è la somma delle ricompense scontate:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (1)$$

L'agente segue una **policy** (politica) $\pi(a|s)$, ovvero la probabilità di scegliere l'azione a nello stato s . La Funzione Valore (Value Function) stima quanto è "buono" uno stato:

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \quad (2)$$

2 L'Algoritmo PPO (Proximal Policy Optimization)

L'algoritmo Proximal Policy Optimization (PPO) è un metodo di tipo *Policy Gradient* introdotto da OpenAI, appartenente alla famiglia degli algoritmi *Actor-Critic*.

Nei metodi standard di policy gradient, aggiornamenti troppo ampi dei pesi della rete neurale possono destabilizzare l'apprendimento. PPO risolve questo problema limitando (clipping) la variazione della policy ad ogni step di aggiornamento.

2.1 La Funzione Obiettivo di PPO

PPO calcola il rapporto tra la nuova policy e la vecchia policy:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (3)$$

La funzione obiettivo *Clipped Surrogate Objective* è definita come:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right] \quad (4)$$

dove $\hat{A}_t$ è la funzione di Vantaggio (Advantage), che indica quanto un'azione è stata migliore rispetto alla media, e ϵ è un iperparametro di clipping (tipicamente 0.2).

3 Il Progetto Snake RL: Analisi dell'Implementazione

Il progetto fornito implementa una variante del classico gioco Snake, addestrato tramite PPO utilizzando la libreria Stable-Baselines3. Il progetto è suddiviso in vari file sorgenti.

3.1 Analisi dei File Sorgenti

- **snake_env.py**: Definisce l'ambiente di gioco ereditando da `gym.Env`. Gestisce le dinamiche spaziali (la griglia 800×490) e lo stato. L'osservazione (stato) è un vettore di 28 valori (distanze da muri, mela e corpo in 8 direzioni, più la codifica one-hot della direzione attuale). Le azioni possibili sono 4 (Su, Giù, Sinistra, Destra). Il sistema di reward shaping è cruciale: +10 per la mela mangiata, +0.015 per l'avvicinamento, -0.015 per l'allontanamento, -0.005 come malus temporale, -1.5 per collisione con i muri e -2.0 per autoscontro. Contiene anche le logiche di salvataggio dello storico delle statistiche su un file JSON.
- **train.py**: Configura e lancia l'addestramento. Crea l'ambiente, definisce una schedule lineare per il Learning Rate, e inizializza il modello PPO con i parametri specificati (architettura della rete, `gamma`, `ent_coef`, ecc.). Gestisce anche il caricamento del modello esistente per continuare l'addestramento senza perdere progressi.
- **main.py**: Contiene il loop principale che unisce il motore grafico Pygame e l'inferenza del modello. Permette di giocare in due modalità: manuale (Classico) o supervisionando l'agente (Addestramento). Utilizza il multiprocessing per lanciare la UI Tkinter in modo da non bloccare il rendering.
- **ui.py**: Contiene tutte le funzioni accessorie per disegnare a schermo il menu, il game over, i pannelli di stato, la finestra popup informativa e i grafici matplotlib-like renderizzati direttamente su Pygame (andamento mele mangiate e passi).

4 Parametri del Modello e Statistiche

Il pannello di monitoraggio (visibile nell'immagine allegata "statistiche.png") è gestito tramite Tkinter e riporta tutti gli iperparametri chiave dell'addestramento. Ecco la loro spiegazione dettagliata:

- **Algoritmo: PPO (Stable-Baselines3)**: L'algoritmo scelto. Molto robusto e adatto per ambienti complessi.
- **Learning Rate: 3e-4 (Lineare)**: Il tasso di apprendimento indica la dimensione dei passi di aggiornamento dei pesi. Uno scheduling lineare significa che parte da 0.0003 e decresce gradualmente fino a zero man mano che l'addestramento converge, stabilizzando il comportamento dell'agente nelle fasi finali.
- **Gamma (Discount): 0.999**: Valore altissimo che fa sì che l'agente guardi molto avanti nel futuro. Avere un $\gamma = 0.999$ in Snake è essenziale perché i veri premi (mangiare molte mele e sopravvivere) richiedono pianificazioni lunghe e complesse.
- **Entropy Coef: 0.01**: Determina quanta importanza dare all'entropia (randomness) delle azioni. Un valore di 0.01 incentiva l'agente a esplorare percorsi alternativi, prevenendo la convergenza prematura verso policy subottimali (es. girare in tondo continuamente).
- **Architettura Rete: pi: [256, 256], vf: [256, 256]**: Rappresenta la struttura dei Multilayer Perceptron (MLP). Vi sono due reti gemelle a 2 strati nascosti da 256 neuroni ciascuno: **pi** (l'Attore) che decide quale azione prendere, e **vf** (il Critico / Value Function) che valuta la bontà dello stato in cui ci si trova.
- **N-Steps: 2048**: Indica quanti passi l'agente deve effettuare nell'ambiente prima di aggiornare i pesi della rete neurale.
- **Batch Size: 64**: La dimensione dei mini-batch estratti dai 2048 passi raccolti per compiere gli aggiornamenti con discesa del gradiente.
- **Epoche: 10**: Numero di volte in cui la rete neurale viene ri-addestrata sull'intero buffer di 2048 step ad ogni aggiornamento.
- **Episodi Totali / Step Totali Ambienti**: Dati di diagnostica che indicano quante partite sono state giocate e quanti movimenti totali (frame) ha elaborato l'agente nel corso dell'apprendimento.
- **Premio RL (Episodio Corrente)**: Somma in tempo reale delle ricompense ottenute nell'episodio in corso (calcolate secondo i valori in `snake_env.py`).